

Edge Detection (Part 2) and Corner Detection

DO Thanh Ha

Informatics Departement
Mathematics - Mechanics - Informatics
VNU University of Science

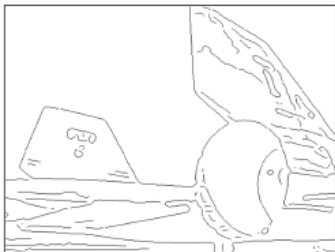
- Edge Detection (Part 2)
- Corner Detection

Recall: Edge Detection

- Image processing task that finds edges and contours in images
- Edges so important that human vision can reconstruct edge lines



(a)



(b)

Characteristics of an Edge

- Real (non-ideal) edge is a slightly blurred step function
- Edges can be characterized by high value first derivative

$$f'(x) = \frac{df}{dx}(x)$$

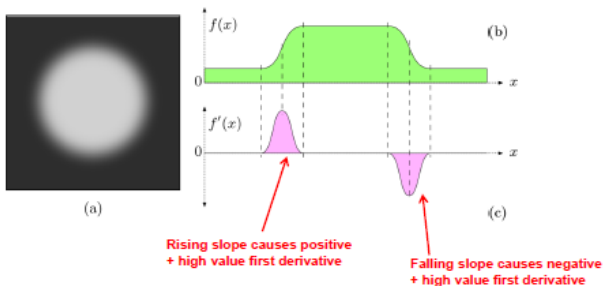


Image Gradient

- Image is 2D discrete function

$$\frac{\partial I}{\partial u}(u, v) \text{ and } \frac{\partial I}{\partial v}(u, v)$$

- Image derivatives in horizontal and vertical directions
- Image gradient at location (u, v)

$$\Delta I(u, v) = \begin{pmatrix} \frac{\partial I}{\partial u}(u, v) \\ \frac{\partial I}{\partial v}(u, v) \end{pmatrix}$$

- Gradient magnitude

$$|\Delta I|(u, v) = \sqrt{\left(\frac{\partial I}{\partial u}(u, v)\right)^2 + \left(\frac{\partial I}{\partial v}(u, v)\right)^2}$$

- Magnitude is invariant under image rotation, used in edge detection

Gradient-Based Edge Detection

- Compute image derivatives by convolution

$$D_x(u, v) = H_x * I \text{ and } D_y(u, v) = H_y * I$$

- Compute edge gradient magnitude

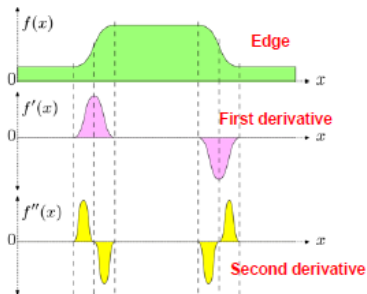
$$E(u, v) = \sqrt{(D_x(u, v))^2 + (D_y(u, v))^2}$$

- Compute edge gradient direction

$$\Phi(u, v) = \tan^{-1}\left(\frac{D_y(u, v)}{D_x(u, v)}\right) = \arctan(D_x(u, v), D_y(u, v))$$

Other Edge Operators

- Problem with edge operators based on first derivatives:
 - Edge is proportional to underlying intensity transition
 - Edges may be difficult to localize precisely
- Solution? Use second derivative
- Recall: An edge corresponds to a zero crossing of the 2nd derivative noise, pre-smoothing filters used first



Edges at Different Scales

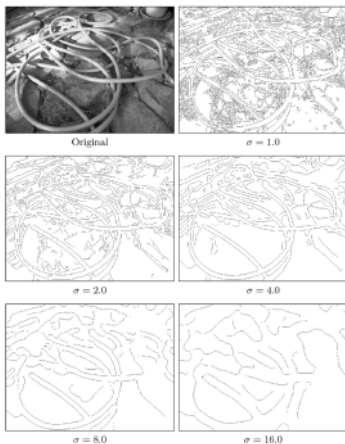
- Simple edge operators deviate from human perception in main way: Edges exist at multiple scales
- **Hierarchical or pyramid techniques:**
 - For each image position (u, v) , apply edge detection filters at multiple scales
 - Use most dominant edge/scale detected (if any)

Canny Edge Detector

- Popular edge detector that operates at different scales, then combines results into common edge map. Tries to:
 1. Minimize number of false edge points
 2. Achieve good localization of edges
 3. Deliver only a single mark on each edge
- Essentially gradient based using zero crossings of second derivative

Canny Edge Detector

- Typically, a single scale implementation (1 image) used with adjustable filter radius (smoothing parameter σ)



Resulting edge σ maps for different settings of the smoothing
(scale) parameter σ

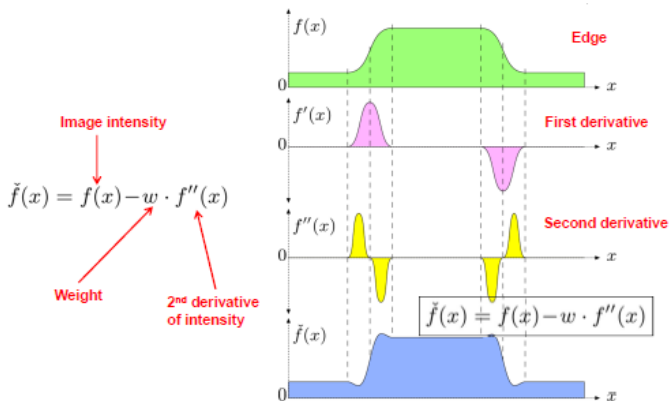
From Edges to contours

- Edge detection yields at each image position (u, v) : Edge strength + orientation
- How can this information be used to detect contours?
- **Contour following**: Starting from image point with high edge strength, follow edge iteratively till the 2 traces meet and a closed contour is formed
- **Several obstacles make this impractical and rarely used**:
 - Edges may end in regions of vanishing intensity gradient
 - Crossing edges lead to ambiguities
 - Contours may branch into several directions

- Usually after calculating edge strengths, we just make binary decision whether point is valid edge
- **Most common approach**: generate **edge map** by applying threshold (fixed or adaptive) to edge strengths calculated by edge operator
- In practice edge maps seldom contain perfect contours
- Edge map frequently has small, unconnected contour fragments interrupted by positions of insufficient strength

- Blurring may occur during image scanning or scaling
- Sharpening reduces effects of blurring
- How? Amplify high frequency components
- High frequencies occur at edges
- Two main approaches:
 - Using Laplace filter
 - Unsharp masking

Edge Sharpening using Laplace Filter



Laplace Operator

- 2D Laplace operator: combines 2nd derivatives in horizontal and vertical directions
- Laplace operator defined as:

$$(\Delta^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$$

- Digital approximation of laplacian is:

$$\Delta^2 f(x, y) = [f(x+1, y) - f(x, y)] - [f(x, y) - f(x-1, y)] + \\ [f(x, y+1) - f(x, y)] - [f(x, y) - f(x, y-1)]$$

$$\rightarrow \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

- Laplace operator defined as:

$$(\Delta^2 f)(x, y) = \frac{\partial^2 f}{\partial^2 x}(x, y) + \frac{\partial^2 f}{\partial^2 y}(x, y)$$

- 1D filters that estimate 2nd derivatives along x and y directions

$$\frac{\partial^2 f}{\partial^2 x} = H_x^L = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \text{ and } \frac{\partial^2 f}{\partial^2 y} = H_y^L = \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}$$

$$H^L = H_x^L + H_y^L = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

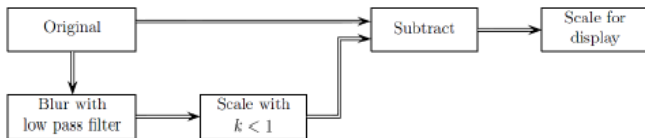
To perform actual sharpening:

- First apply Laplace filter to Image I
- Subtract fraction of result from original image

$$\bar{I} \leftarrow I - w \cdot (H^L * I)$$

Unsharp Masking (USM)

- A technique for edge sharpening: combine image with smoothed (blurred) version of image



- Subtract smooth version (Gaussian smoothing) from image to obtain enhanced edge mask

$$M \leftarrow I - (I * H) = I - \bar{I}$$

- Add the mask to image with a weight

$$I' \leftarrow I + a \cdot M$$

- Together:

$$I' \leftarrow I + a \cdot (I - \bar{I}) = (1 + a) \cdot I - a \cdot \bar{I}$$

- Advantages of USM over Laplace filter
 - Reduced noise sensitivity due to smoothing
 - Improved control through parameters σ and a

Unsharp Masking (USM)

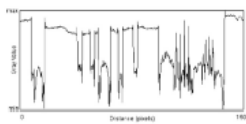
original



(a) Original



(b)



(c)

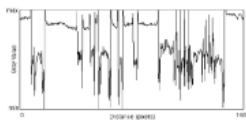
Results of USM
($\sigma = 2.5$)



(d) $\sigma = 2.5$



(e)



(f)

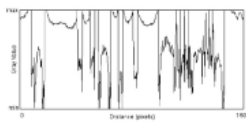
Results of USM
($\sigma = 10.0$)



(g) $\sigma = 10.0$



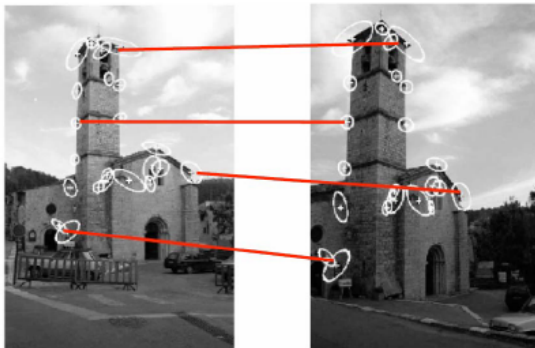
(h)



(i)

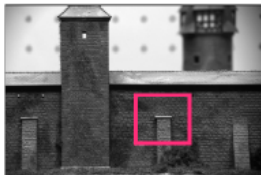
Why Corner Detection?

- Corners are robust, used in computer vision for matching same point in multiple images (e.g. stereo left + right image)



Motivation: Patches Matching

- Consider small squares called patches of same size in both images



- The task? Find most similar patch in second image



?
=



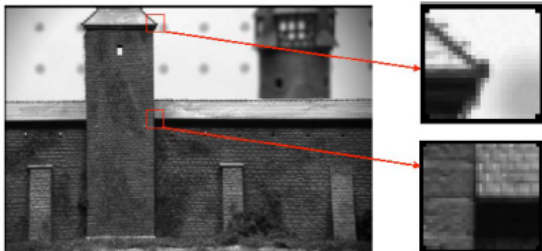
Some Patches is Better

- More robust to use distinctive patches
- Example: Don't use patches similar to many patches in image 2 (ambiguous)



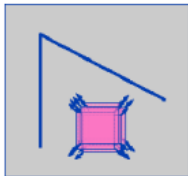
Corners are Robust “Features”

- Corners are unique, match patches with corners
- What are corners? Junctions of contours
- Corners appear as large changes in intensity in different viewpoints (stable/unique)

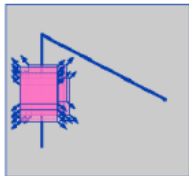


Corner Points: Basic Idea

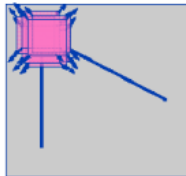
- Patch at corner: shifting window in any direction yields large change in intensity
- Is patch at corner? shift window in multiple directions, if large intensity changes, patch is a corner



Flat region: no intensity change in all directions



Edge: no change along edge directions

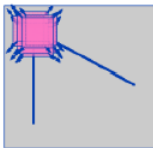


Corner: significant intensity change in many directions

Harris corner detector gives mathematical approach for determining which case holds

Harris Detector: The Mathematics

- Shift patch by $[u,v]$ and compute change in intensity
- Change of intensity for shift $[u,v]$
- For nearly flat regions: $I(x+u, y+v) - I(x, y)$ almost 0
- For distinctive (corner) : $I(x+u, y+v) - I(x, y)$ almost large. Hence ... Corner? Patches where $E(u, v)$ is LARGE

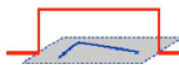


$$E(u, v) = \sum w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

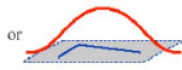
Diagram illustrating the components of the Harris detector equation:

- Window function $w(x, y)$
- Shifted intensity $I(x+u, y+v)$
- Intensity $I(x, y)$

Window function $w(x, y) =$



1 in window, 0 outside



Gaussian

Taylor Series Expansion

- Taylor Series: $f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \frac{f'''(x_0)}{3!}(x - x_0)^3 + \dots$



$$f(x + u, y + v) = f(x, y) + uf_x(x, y) + vf_y(x, y) + \frac{1}{2!}[u^2 f_{xx}(x, y) + uvf_{xy}(x, y) + v^2 f_{yy}(x, y)] + \dots$$

- First order approximation

$$\longrightarrow [I(x + u, y + v) - I(x, y)]^2 \approx [I(x, y) + ul_x + vl_y - I(x, y)]^2$$

Harris Corner Derivation

■ First order approximation

$$\begin{aligned} \rightarrow \sum [I(x+u, y+v) - I(x, y)]^2 &\approx \sum [I(x, y) + uI_x + vI_y - I(x, y)]^2 \\ &= \sum u^2 I_x^2 + 2uvI_x I_y + v^2 I_y^2 \\ &= \sum (u \quad v) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} \\ &= (u \quad v) \left(\sum \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} \right) \begin{pmatrix} u \\ v \end{pmatrix} \end{aligned}$$

■ For small shifts $[u, v]$, we have following approximation

$$E(u, v) = (u \quad v) M \begin{pmatrix} u \\ v \end{pmatrix} \text{ where } M = \sum_{x,y} w(x, y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

$$M = \sum_{x,y} w(x,y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

- If we compute derivatives $A(u, v)$, $B(u, v)$ and $C(u, v)$:
 - $A(u, v) = I_x^2(u, v)$
 - $B(u, v) = I_x I_y(u, v)$
 - $C(u, v) = I_y^2(u, v)$
- Can express matrix M as

$$M = \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix} = \begin{pmatrix} A & C \\ C & B \end{pmatrix}$$

Harris Corner Detector

- Smooth $A(u, v)$, $B(u, v)$ and $C(u, v)$ individually with linear gaussian filter

$$\bar{M} = \begin{pmatrix} A * H^{G,\sigma} & C * H^{G,\sigma} \\ C * H^{G,\sigma} & B * H^{G,\sigma} \end{pmatrix} = \begin{pmatrix} \bar{A} & \bar{C} \\ \bar{C} & \bar{B} \end{pmatrix}$$

- Since the matrix $\bar{M}$ is symmetric, it can be diagonalized

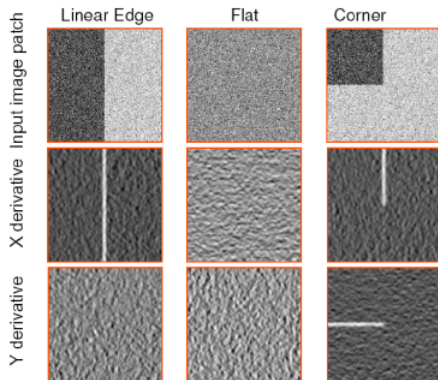
$$\bar{M}' = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$$

where λ_1, λ_2 are eigenvalues of matrix $\bar{M}$;

- Edge: $\lambda_1 \gg \lambda_2$ or $\lambda_1 \ll \lambda_2$
- Corner: λ_1, λ_2 are large. E increases in all directions
- Flat region: λ_1, λ_2 are small, E is almost constant in all directions

Harris Corner Detection Intuition

■ Example: Cases and 2D Derivatives



Harris Corner Detector

$$\bar{M}' = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$$

- Function for corner detection

$$Q(u, v) = \det(\bar{M}) - \alpha \cdot (\text{trace}(\bar{M}))^2$$

- An image location (u, v) is candidate for corner point when $Q(u, v) > t_H$
- t_H is threshold selected based on image content, typically lies in range 10,000 – 1,000,000
- Detected corners inserted into set and sorted in descending order (i.e. $q_i \geq q_{i+1}$) based on their corner strength

$$\text{Corners} = [c_1, c_2, \dots, c_N]$$

- Many false corners occur in neighborhood of real corner
- Traverse sorted list, delete false corners towards end of list

- Digital Image Processing (CS/ECE 545), Computer Science Dept. Worcester Polytechnic Institute (WPI)